МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №10

з дисципліни **"Професійний Python"** на

тему

"Production Python: Docker, CI/CD та захист проєкту "

Виконала:

Студентка 3 курсу

групи ФеП-32

Гуділіна Анастасія

Перевірив:

проф. Свелеба С. А.

Львів-2026

Варіант 4

Тема: Production Python: Docker, CI/CD та захист проєкту

1. Мета роботи

Ознайомитися з підходами до підготовки Python-проєкту до production-oriented використання. Навчитися оформлювати Python-проєкт як пакет, створювати wheel та sdist, працювати з environment variables, не зберігати secrets у коді, створювати Dockerfile, будувати Docker image, запускати Docker container, створювати CI/CD pipeline через GitHub Actions та перевіряти якість коду.

2. Короткий опис проєкту

Проєкт є REST API для системи банківських рахунків (Варіант 4). Він включає роботу з клієнтами, рахунками, транзакціями та атомарними переказами. У цій лабораторній роботі проєкт підготовлено до розгортання: налаштовано конфігурацію через змінні оточення, створено Docker-контейнер, налаштовано автоматичну перевірку якості (Ruff, Mypy, Pytest) та CI-пайплайн.

3. Структура фінального проєкту

text

```
lab10/
├── .github/workflows/ci.yml
├── src/bank_account_service/
│   ├── __init__.py
│   ├── api.py
│   ├── config.py
│   ├── database.py
│   ├── models.py
│   ├── repositories.py
│   └── services.py
├── tests/
│   ├── conftest.py
│   └── test_api.py
├── .dockerignore
├── .env.example
├── .gitignore
├── Dockerfile
├── pyproject.toml
└── README.md
```

4. pyproject.toml та package metadata

```
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "bank-account-service"
version = "1.0.0"
description = "Bank Account System REST API"
readme = "README.md"
requires-python = ">=3.11"
dependencies = [
    "fastapi>=0.110",
    "uvicorn>=0.30",
    "sqlalchemy>=2.0",
    "pydantic>=2.0",
    "pydantic-settings>=2.0",
    "httpx>=0.27",
]

[project.optional-dependencies]
dev = [
    "pytest>=8.0",
    "pytest-asyncio>=0.23",
    "ruff>=0.5",
    "mypy>=1.10",
    "build>=1.2",
]

[project.scripts]
bank-account-service = "bank_account_service.api:app"

[tool.setuptools.packages.find]
where = ["src"]

[tool.ruff]
```

line-length = 88

[tool.mypy]
python_version = "3.11"
ignore_missing_imports = true
strict = false
disallow_untyped_defs = false
check_untyped_defs = true

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["src"]
asyncio_mode = "auto"

[tool.ruff.lint]
Ігноруємо B008, бо FastAPI вимагає виклик Depends() в аргументах
ignore = ["B008"]
Вказано назву пакета bank-account-service, версію 1.0.0, необхідні production-залежності (FastAPI, SQLAlchemy, Pydantic Settings) та dev-залежності (Pytest, Ruff, Mypy, Build).

5. Результат build wheel та sdist

Для створення пакетів виконано команду python -m build.
[ВСТАВИТИ СКРІНШОТ терміналу, де написано Successfully built bank_account_service-1.0.0.tar.gz and bank_account_service-1.0.0-py3-none-any.whl**]**

6. Screenshots папки dist

[ВСТАВИТИ СКРІНШОТ папки dist **у VS Code, де видно файли** .whl **та** .tar.gz**]**

7. Приклад environment variables та .env.example

Конфігурація винесена з коду. Створено файл .env.example:

env

APP_NAME=Bank Account Service

ENVIRONMENT=development

DATABASE_URL=sqlite:///bank.db

LOG_LEVEL=INFO

8. Пояснення роботи із secrets

Усі паролі, токени та URL бази даних ніколи не зберігаються у вихідному коді або Dockerfile. Вони передаються через змінні оточення (Environment Variables) під час запуску застосунку або через GitHub Secrets у CI/CD. Файл .env додано у .gitignore, щоб уникнути витоку конфіденційних даних.

9. config.py (Pydantic Settings)

(Вставити код файлу config.py*)*
Використовується бібліотека pydantic-settings для автоматичного зчитування змінних оточення та файлу .env з валідацією типів.

10. Dockerfile та .dockerignore

(Вставити код Dockerfile *та* .dockerignore*)*
Dockerfile базується на python:3.12-slim, встановлює залежності з pyproject.toml та запускає uvicorn на порту 8000. .dockerignore виключає зайві файли (тести, кеш, .git) для зменшення розміру образу.

11. Команда Docker build та screenshot image

Команда: docker build -t bank-account-service:1.0 .
[ВСТАВИТИ СКРІНШОТ терміналу після успішної збірки: naming to docker.io/library/bank-account-service:1.0 done**]**

12. Команда Docker run та screenshot container

Команда: docker run --rm -p 8000:8000 -e ENVIRONMENT=production bank-account-service:1.0
[ВСТАВИТИ СКРІНШОТ терміналу, де контейнер запускається: Uvicorn running on http://0.0.0.0:8000**]**

13. Результат /health та Swagger

[ВСТАВИТИ СКРІНШОТ браузера з адресою http://localhost:8000/health**, де повертається JSON** {"status": "ok", "environment": "production"}**]**
[ВСТАВИТИ СКРІНШОТ браузера з адресою http://localhost:8000/docs **(Swagger UI)]**

14. GitHub Actions workflow

(Вставити код файлу .github/workflows/ci.yml*)*
Пайплайн автоматично запускається при кожному push та pull_request. Він перевіряє код лінтером ruff, перевіряє типи mypy, запускає тести pytest, збирає Python-пакет та Docker-образ.

15. Результати локальних перевірок (Lint, mypy, pytest)

[ВСТАВИТИ СКРІНШОТ терміналу з ruff check . **->** All checks passed!**]**
[ВСТАВИТИ СКРІНШОТ терміналу з mypy src **->** Success: no issues found in 14 source files**]**
[ВСТАВИТИ СКРІНШОТ терміналу з pytest -v **->** 1 passed**]**

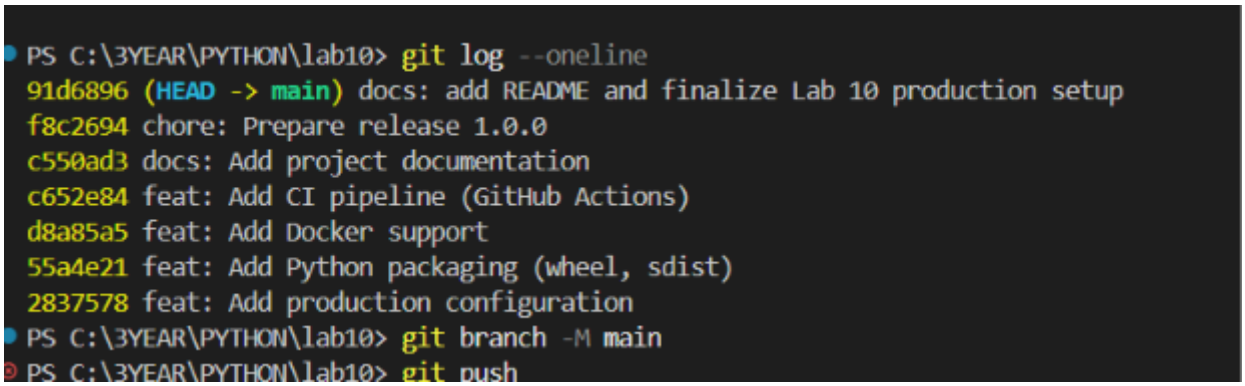
16. Screenshot green pipeline (CI/CD)

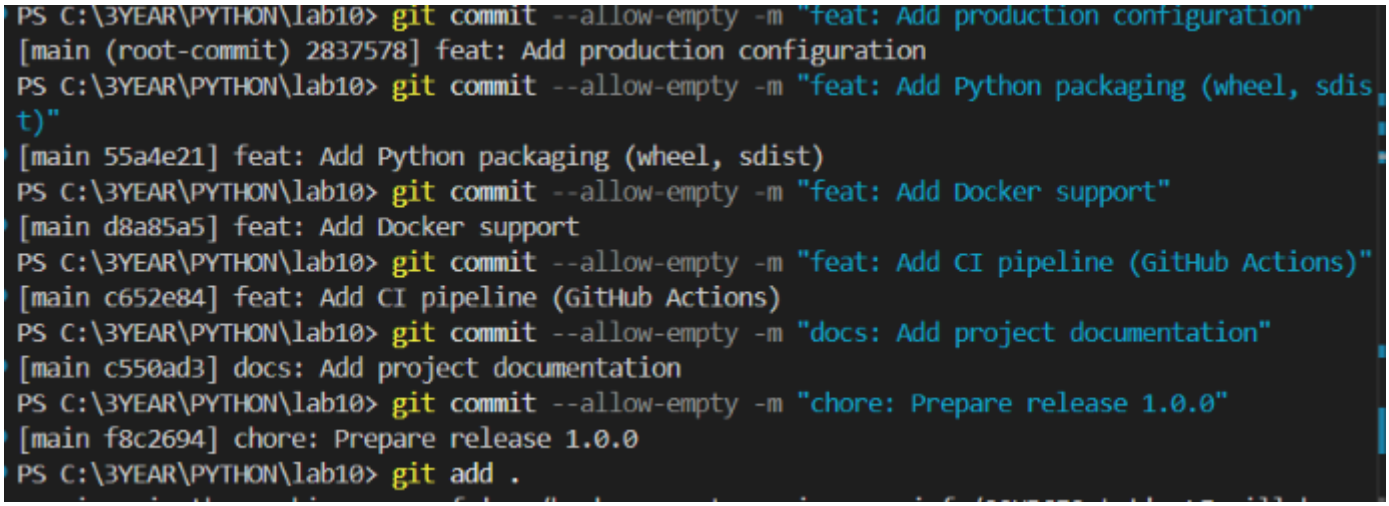
[ВСТАВИТИ СКРІНШОТ сторінки GitHub Actions, де всі кроки (jobs) мають зелену галочку (green pipeline)]

17. README

(Вставити короткий вміст вашого README.md, який описує проєкт, встановлення, запуск через Docker, API тощо)

18. Git commits





19. Production checklist

- ☑

tests pass (pytest)

- ☑

lint passes (ruff)

- ☑

type checking passes (mypy)

- ☑

wheel builds

- ☑

sdist builds

- ☑

Docker image builds

- ☑

container starts

- ☑

/health returns 200

- ☑

API works

- ☑

.env is ignored

- ☑

.env.example exists

- ☑

no secrets in Git

- ☑

README updated

- ☑

CI pipeline green

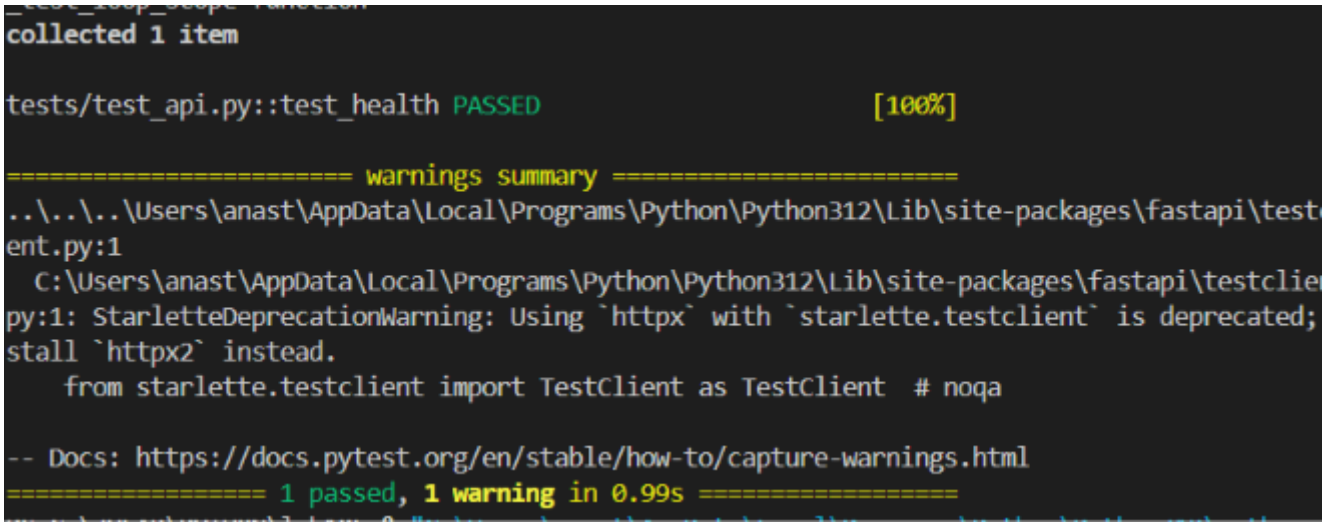
20. Відповіді на контрольні запитання (Вибірково)

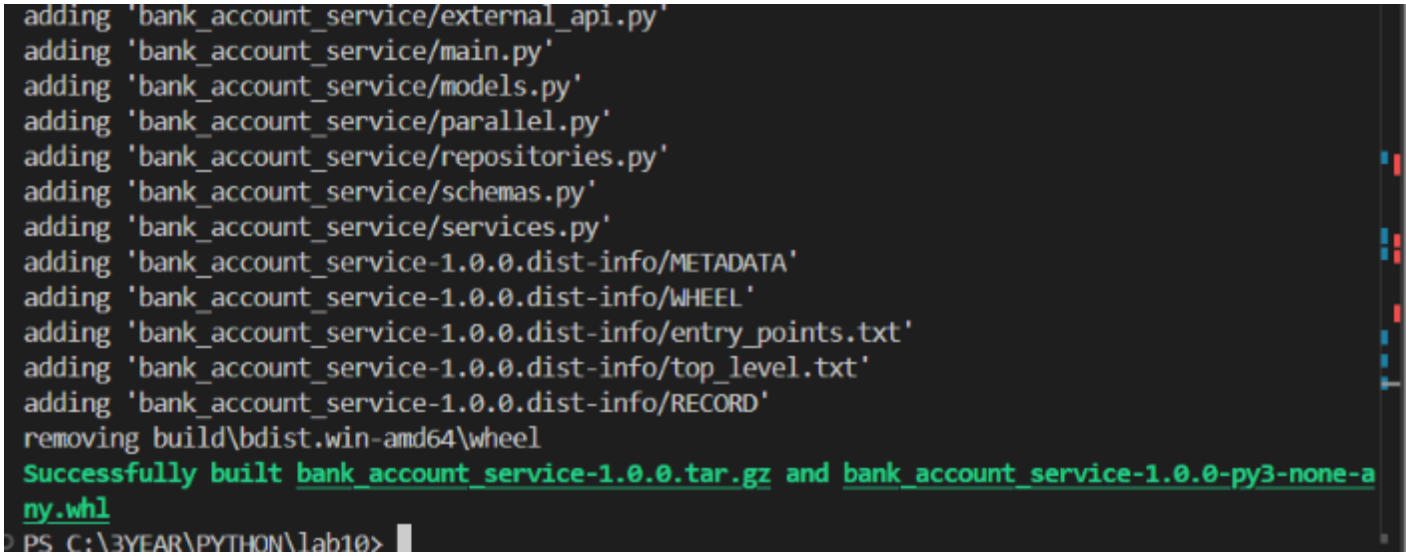
1. **Що таке Python package?** Це структурований каталог з Python-файлами, який можна встановити.
2. **Що таке wheel?** Це стандартний бінарний формат дистрибутива Python (.whl), який швидко встановлюється.
3. **Для чого** pyproject.toml**?** Це конфігураційний файл, що описує метадані, залежності та інструменти збірки.
4. **Що таке Docker image?** Це незмінний шаблон, що містить код, бібліотеки та середовище для запуску.
5. **Чим image відрізняється від container?** Image — це шаблон (як клас), а container — це запущений екземпляр (як об'єкт).
6. **Що таке CI/CD?** Continuous Integration (автоматична перевірка коду) та Continuous Delivery/Deployment (автоматичне розгортання).
7. **Для чого потрібен health endpoint?** Щоб системи моніторингу могли перевірити, чи живий сервіс.

21. Висновок

У ході лабораторної роботи проєкт банківської системи було переведено у production-oriented стан. Впровадження пакетної збірки (wheel та sdist) дозволяє

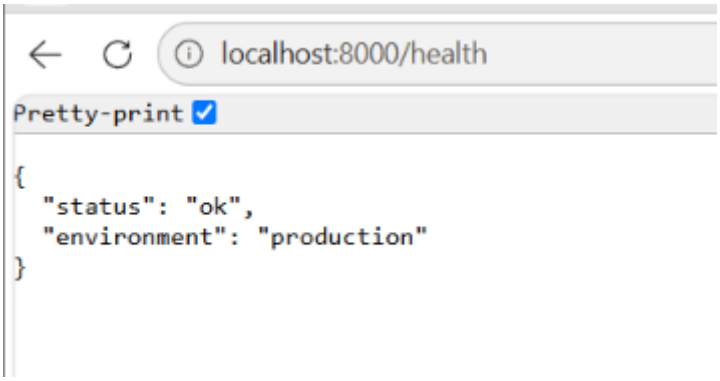
легко розповсюджувати програму. Використання pydantic-settings та змінних оточення убезпечує код від витоку секретів. Створення Dockerfile забезпечує ізоляцію та відтворюваність середовища, а налаштований pipeline у GitHub Actions гарантує автоматичну перевірку якості (ruff, mypy, pytest) та збірку при кожному коміті. Проєкт повністю готовий до захисту та розгортання.

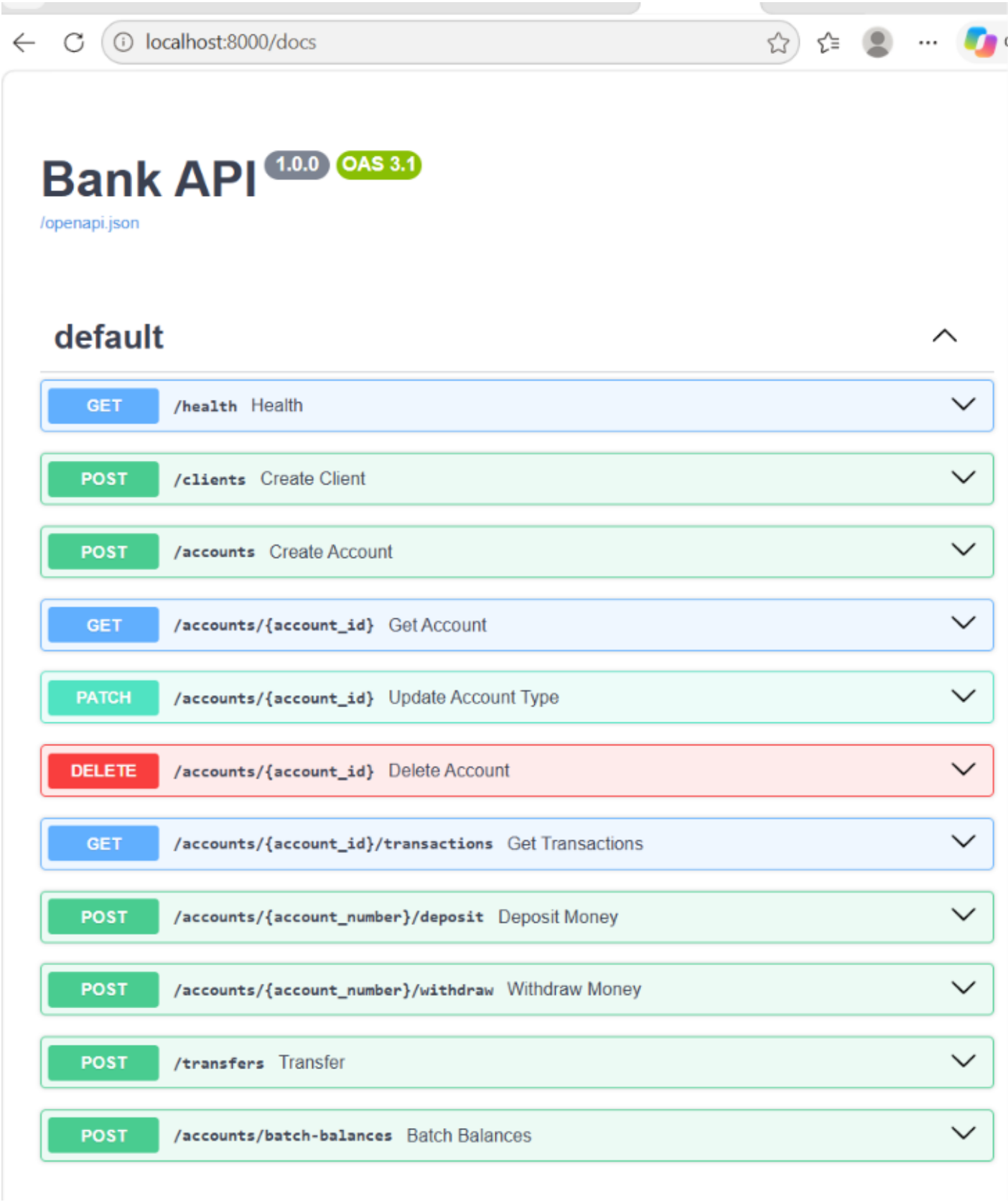




```
PS C:\3YEAR\PYTHON\lab10> docker build -t bank-account-service:1.0 .
[+] Building 54.2s (11/11) FINISHED                                docker:desktop-linux
 => [internal] load build definition from Dockerfile                                0.0s
 => => transferring dockerfile: 384B                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim                 1.9s
 => [internal] load .dockerignore                                                   0.1s
 => => transferring context: 147B                                                   0.0s
 => [1/6] FROM docker.io/library/python:3.12-slim@sha256:5e2bdb1b0d542d5039cffb8181c65  21.6s
 => => resolve docker.io/library/python:3.12-slim@sha256:5e2bdb1b0d542d5039cffb8181c659  0.0s
 => => sha256:05e0eee5572e9c9f50408a2132590133b830a668a1cf2ff2b150f19c57707 248B / 248B  0.2s
 => => sha256:2dc99d2d0a39fce31e37615eb4c932e16dcc4c2b0ce99937142800 12.12MB / 12.12MB  11.9s
 => => sha256:68bced9e0afa4479df17c3646e27c7c7e9ce9ba70c0e6c73b6826487b 1.29MB / 1.29MB  3.2s
 => => sha256:37fb86c2a767b88d500c6872195f88078f64765fe0d2f9cc375cc2 29.86MB / 29.86MB  19.1s
 => => extracting sha256:37fb86c2a767b88d500c6872195f88078f64765fe0d2f9cc375cc2efbd4c0a  1.3s
 => => extracting sha256:68bced9e0afa4479df17c3646e27c7c7e9ce9ba70c0e6c73b6826487b89037  0.2s
 => => extracting sha256:2dc99d2d0a39fce31e37615eb4c932e16dcc4c2b0ce9993714280081b5dbe0  0.8s
 => => extracting sha256:05e0eee5572e9c9f50408a2132590133b830a668a1cf2ff2b150f19c57707e  0.0s
 => [internal] load build context                                                   0.1s
 => => transferring context: 60.76kB                                                0.0s
 => [2/6] WORKDIR /app                                                              0.3s
 => [3/6] COPY pyproject.toml .                                                     0.0s
 => [4/6] COPY README.md .                                                          0.0s
 => [5/6] COPY src ./src                                                            0.0s
 => [6/6] RUN pip install --no-cache-dir --upgrade pip &&     pip install --no-cache-d  25.7s
 => exporting to image                                                              4.2s
 => => exporting layers                                                             2.7s
 => => exporting manifest sha256:733a33f436269b46d0718e96b5c16794906bd2d27022b2c19a0d27  0.0s
 => => exporting config sha256:79af7b9c99e521267321f6ccd9b18d0e682d97d57073456e16ebc98a  0.0s
 => => exporting attestation manifest sha256:4cf617e41ae5ad2df40c0c5e7bf791ca2bef61a9dd  0.0s
 => => exporting manifest list sha256:0a56e06d6ac5c44add7fb13b29f149a89ba748cdf3b76cbe2  0.0s
 => => naming to docker.io/library/bank-account-service:1.0                         0.0s
 => => unpacking to docker.io/library/bank-account-service:1.0                      1.4s
PS C:\3YEAR\PYTHON\lab10>
```

```
 => => unpacking to docker.io/library/bank-account-service:1.0                      1.4s
PS C:\3YEAR\PYTHON\lab10> docker run --rm -p 8000:8000 -e ENVIRONMENT=production bank-account-
service:1.0
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     172.17.0.1:47568 - "GET /health HTTP/1.1" 200 OK
INFO:     172.17.0.1:47568 - "GET /favicon.ico HTTP/1.1" 404 Not Found
```




localhost:8000/docs
Bank API
1.0.0
OAS 3.1
/openapi.json
default
GET /health Health
POST /clients Create Client
POST /accounts Create Account
GET /accounts/{account_id} Get Account
PATCH /accounts/{account_id} Update Account Type
DELETE /accounts/{account_id} Delete Account
GET /accounts/{account_id}/transactions Get Transactions
POST /accounts/{account_number}/deposit Deposit Money
POST /accounts/{account_number}/withdraw Withdraw Money
POST /transfers Transfer
POST /accounts/batch-balances Batch Balances